



Institución
Universitaria
Reacreditada en Alta Calidad

ESTRUCTURA DE DATOS Y »»» LABORATORIO «««

580506004-8

William Giraldo Escobar
Facultad de Ingenierías
Departamento de Sistemas de Información



LISTAS

Una lista es una estructura de datos que se utiliza para almacenar una *colección ordenada* de elementos. Estos elementos pueden ser de cualquier tipo, como números, cadenas de texto, objetos, otras listas, etc.





LISTAS

Puedes crear una lista en Python utilizando corchetes [] y separando los elementos con comas.

```
Lista = [ elemento 1, elemento 2, elemento 3]
```

LISTAS

A cada elemento de una lista le corresponde un *índice*.

El índice es la *posición* que ocupa ese elemento dentro de la lista.

Lista = [elemento 1, elemento 2, elemento 3]

0

1

2

LISTAS

NOTA:

Los índices en Python comienzan en 0, por lo que el primer elemento de la lista tiene un índice de 0, el segundo tiene un índice de 1 y así sucesivamente.

```
Lista = [ elemento 1, elemento 2, elemento 3]
```

0

1

2

LISTAS

Los elementos almacenados dentro de una lista pueden ser de cualquier tipo.

Todos los elementos pueden ser de la misma clase o de clases distintas.

```
Lista1 = [ 8 , - 9 , 4 ]
```

```
Lista2 = [ 'hola' , True , 0.53 ]
```


CREACIÓN DE LISTAS

Puedes crear una lista en Python de varias formas:

- Lista vacía

```
mi_lista = []
```

- Lista con elementos

```
mi_lista = [1, 2, 3, 4, 5]
```

- Lista de diferentes tipos de datos

```
mi_lista = [1, "Ana", 3.0, True]
```

ACCESO A ELEMENTOS DE UNA LISTA

Puedes acceder a los elementos de una lista utilizando *índices*. Los índices en Python *comienzan* desde 0.

```
mi_lista = [10, 20, 30, 40, 50]
```

```
primer_elemento = mi_lista[0]
```

```
print(primer_elemento)
```


ACCESO A ELEMENTOS DE UNA LISTA

Puedes acceder a los elementos de una lista utilizando *índices*. Los índices en Python *comienzan* desde 0.

```
mi_lista = [10, 20, 30, 40, 50]
```

```
primer_elemento = mi_lista[3]
```

```
print(primer_elemento)
```

ACCESO A ELEMENTOS DE UNA LISTA

Podemos acceder a los elementos yendo de atrás hacia adelante

```
mi_lista = [10, 20, 30, 40, 50]
```

```
primer_elemento = mi_lista[-1]
```

```
print(primer_elemento)
```

ACCESO A ELEMENTOS DE UNA LISTA

Forma directa

```
mi_lista = [10, 20, 30, 40, 50]
```

```
Print(mi_lista[-1])
```

```
Print(mi_lista[4])
```

ACCESO A ELEMENTOS DE UNA LISTA - CICLOS

Puedes imprimir los elementos de una lista utilizando un ciclo for en Python.

```
mi_lista = [1, 2, 3, 4, 5]
```

```
for elemento in mi_lista:  
    print(elemento)
```

ACCESO A ELEMENTOS DE UNA LISTA - CICLOS

Este código imprimirá cada elemento de la lista `mi_lista` en una línea separada. El bucle `for` recorre la lista uno por uno y la variable `elemento` toma el valor de cada elemento en cada iteración del ciclo.

```
mi_lista = ["Luis", 2.8, True, "carro", 5]

for elemento in mi_lista:
    print(elemento)
```

ACCESO A ELEMENTOS DE UNA LISTA - CICLOS

Si deseas imprimir los elementos en una sola línea separados por un espacio, puedes hacerlo así:

```
mi_lista = [1, 2, 3, 4, 5]

for elemento in mi_lista:
    print(elemento, end=' ')
```

ACCESO A ELEMENTOS DE UNA LISTA - CICLOS

Este código imprimirá los elementos de la lista en la misma línea y separados por espacios. El argumento `end=' '` en la función `print` especifica que después de imprimir cada elemento, se debe usar un espacio en lugar del salto de línea predeterminado.

```
mi_lista = [1, 2, 3, 4, 5]

for elemento in mi_lista:
    print(elemento, end=' ')
```


ACCESO A ELEMENTOS DE UNA LISTA - CICLOS

Para recorrer los elementos de una lista en un ciclo de atrás hacia adelante en Python, puedes utilizar un bucle for con la función *reversed()*

```
mi_lista = [1, 2, 3, 4, 5]
```

```
for elemento in reversed(mi_lista):  
    print(elemento)
```

ACCESO A ELEMENTOS DE UNA LISTA - CICLOS

Para recorrer los elementos de una lista en un ciclo de atrás hacia adelante en Python, puedes utilizar un bucle for con la función *reversed()*

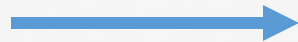
```
mi_lista = ["Luis", 2.8, True, "carro", 5]

for elemento in reversed(mi_lista):
    print(elemento)
```

OPERACIONES CON LISTAS

Agregar un elemento al final de una lista

```
mi_lista.append(60)  
print(mi_lista)
```

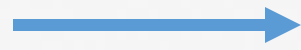


Agregamos el elemento 60
al final de la lista

OPERACIONES CON LISTAS

Agregar un elemento al final de una lista

```
mi_lista.append('hola')  
print(mi_lista)
```



Agregamos el elemento
'hola' al final de la lista

OPERACIONES CON LISTAS

Agregar varios elementos al final de una lista: creamos dos listas y las unimos en una sola

```
mi_lista = [1, 2, 3]
elementos_a_agregar = [4, 5, 6]

mi_lista.extend(elementos_a_agregar)

print(mi_lista) # Esto imprimirá: [1, 2, 3, 4, 5, 6]
```

OPERACIONES CON LISTAS

Agregar varios elementos al final de una lista: creamos dos listas y las unimos en una sola

```
mi_lista = [1, 2, 3]
elementos_a_agregar = [4, 5, 6]

mi_lista += elementos_a_agregar

print(mi_lista) # Esto imprimirá: [1, 2, 3, 4, 5, 6]
```

OPERACIONES CON LISTAS

Agregar varios elementos al final de una lista: creamos dos listas y las unimos en una sola

```
lista1 = [1, 2, 3]
```

```
lista2 = [4, 5, 6]
```

```
concatenada = lista1 + lista2
```

```
print(concatenada) # Esto imprimirá: [1, 2, 3, 4, 5, 6]
```

OPERACIONES CON LISTAS

Insertar un elemento en una posición fija

```
lista1 = [1, 2, 3, 4, 5, 6]
```

```
lista1.insert(2, 25) # Inserta 25 en la posición 2
```


OPERACIONES CON LISTAS

Insertar un elemento en una posición fija

```
lista1 = [1, 2, 3, 4, 5, 6]
```

```
lista1.insert(4, 'casa') # Inserta 'casa' en la posición 4
```

OPERACIONES CON LISTAS

Insertar un elemento en una posición fija

```
lista1 = [1, 2, 3, 4, 5, 6]
```

```
lista1[2]=True # Inserta un True en la posición 2
```

OPERACIONES CON LISTAS

Eliminar un elemento

```
lista1 = [100, 20, 30, 20, 50, 60]
```

```
mi_lista.remove(20) # Elimina el primer 20 que encuentre
```

OPERACIONES CON LISTAS

Eliminar un elemento

```
lista1 = ['casa', 20, 'apartamento', 20, 'casa', 60]
```

```
lista1.remove('casa') # Elimina el primer 'casa' que encuentre
```

OPERACIONES CON LISTAS

Eliminar un elemento según su posición

```
mi_lista = [1, 2, 3, 4, 5]
```

```
del mi_lista[2] # elimina el elemento en la posición 2
```

```
print(mi_lista) # Esto imprimirá: [1, 2, 4, 5]
```

OPERACIONES CON LISTAS

Eliminar un elemento: El método `pop()` elimina un elemento en un índice específico y lo devuelve. Si no se especifica un índice, elimina y devuelve el último elemento de la lista.

```
mi_lista = [1, 2, 3, 4, 5]
```

```
elemento_eliminado = mi_lista.pop(2)
```

```
print(mi_lista) # Esto imprimirá: [1, 2, 4, 5]
```

```
print(elemento_eliminado) # Esto imprimirá: 3
```

OPERACIONES CON LISTAS

Obtener la longitud de una lista: es decir, saber cuántos elementos hay en ella

```
mi_lista = [1, 2, 3, 4, 5]
```

```
longitud = len(mi_lista)
```

```
Print(longitud)
```

OPERACIONES CON LISTAS - ACTIVIDAD

Crear y manipular una lista

Crea una lista vacía llamada `mi_lista` y luego realiza las siguientes operaciones:

- Agrega los números del 1 al 5 a `mi_lista`.
- Inserta el número 10 en la posición 2.
- Elimina el número 3 de la lista.
- Imprime el último elemento de la lista.
- Imprime la longitud de la lista.

OPERACIONES CON LISTAS

Verificar si un elemento está en la lista

```
lista1 = ['casa', True, 'apartamento', 8.75, 'eficicio', 60]
```

```
elemento = 'casa'
```

```
if elemento in lista1:
```

```
    print("El elemento está en la lista")
```

```
else:
```

```
    print("El elemento no está en la lista")
```

OPERACIONES CON LISTAS

Ordenar una lista: de forma ascendente

```
lista1=['carlos','tomás','andrea','susana']
```

```
lista1.sort() # Ordena la lista de forma ascendente  
print(lista1)
```

OPERACIONES CON LISTAS

Ordenar una lista: de forma ascendente

```
lista1=[80 , -2 , 0 , 2000 , 2.5]
```

```
lista1.sort() # Ordena la lista de forma ascendente  
print(lista1)
```

OPERACIONES CON LISTAS

Ordenar una lista: de forma descendente

```
lista1=['carlos','tomás','andrea','susana']
```

```
lista1.sort(reverse=True) # Ordena la lista de forma descendente  
print(lista1)
```

OPERACIONES CON LISTAS

Ordenar una lista: de forma descendente

```
lista1=[80 , -2 , 0 , 2000 , 2.5]
```

```
lista1.sort(reverse=True) # Ordena la lista de forma descendente  
print(lista1)
```



OPERACIONES CON LISTAS

Copiar una lista : evitar cambios no deseados

```
copia_lista = mi_lista.copy()
```

OPERACIONES CON LISTAS

Rebanado (slicing) de listas:

Puedes obtener porciones de una lista utilizando la notación de rebanado.

```
mi_lista = [10, 20, 30, 40, 50]
```

```
# Obtener una sublista desde el índice 1 hasta el 3 (sin incluir el 3)
```

```
sublista = mi_lista[1:3]
```

```
print(sublista)
```

OPERACIONES CON LISTAS

Rebanado (slicing) de listas:

Puedes obtener porciones de una lista utilizando la notación de rebanado.

```
mi_lista = [10, 20, 30, 40, 50]
```

```
# Desde el inicio hasta el índice 2 (sin incluir el 2)
```

```
sublista = mi_lista[:2]
```

```
print(sublista)
```


OPERACIONES CON LISTAS

Rebanado (slicing) de listas:

Puedes obtener porciones de una lista utilizando la notación de rebanado.

```
mi_lista = [10, 20, 30, 40, 50]  
  
# Desde el índice 2 hasta el final  
  
sublista = mi_lista[2:]  
  
print(sublista)
```

OPERACIONES CON LISTAS - ACTIVIDAD

Comprender el slicing

Dada la lista mi_lista:

```
mi_lista = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
```

Realiza las siguientes operaciones utilizando slicing:

- Crea una nueva lista llamada sublista1 que contenga los elementos desde el segundo al sexto (inclusive).
- Crea una nueva lista llamada sublista2 que contenga los elementos desde el tercer elemento hasta el final.

OPERACIONES CON LISTAS

El slicing te permite especificar un intervalo de índices en el que deseas recorrer la lista y también puedes indicar un paso (stride) para avanzar a través de la lista.

lista[inicio:fin:paso]

- *Inicio*: es el índice desde el que comienza el slicing (incluido).
- *Fin*: es el índice en el que termina el slicing (no incluido).
- *Paso*: es el valor que indica cuántos índices avanzar en cada paso.

OPERACIONES CON LISTAS

Ejemplo: recorre la lista desde el principio hasta el final con un paso de 2, lo que significa que imprimirá los elementos en las posiciones 0, 2, 4, 6 y 8

```
mi_lista = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
# Recorrer la lista de 2 en 2
```

```
for numero in mi_lista[::2]:  
    print(numero)
```

OPERACIONES CON LISTAS - ACTIVIDAD

Bucle y slicing

Dada la siguiente lista:

```
numeros = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Escribe un bucle for que recorra numeros con un paso de 2 y, en cada iteración, imprima el número actual.

El resultado debería ser: 0 2 4 6 8

OPERACIONES CON LISTAS - *enumerate*

En Python, puedes utilizar la función ***enumerate()*** junto con un ciclo, como un bucle for, para recorrer una secuencia (por ejemplo, una lista o una cadena) mientras obtienes tanto el ***valor*** como el ***índice*** de cada elemento en la secuencia.

```
mi_lista = ['manzana', 'banana', 'cereza']  
  
for indice, valor in enumerate(mi_lista):  
    print(f'El elemento en la posición {indice} es {valor}')
```




Institución
Universitaria
Reacreditada en Alta Calidad

¡MUCHAS GRACIAS!

A decorative graphic consisting of several horizontal lines in various colors (yellow, orange, red, purple, blue, green) and a curved line on the right side, all in white.